Sebastian Teslic
10/30/2026
01:198:437

Midterm Follow-up Questions

1. **Overlap enforcement, twice:** the booking procedure checks technician overlap, and a trigger (added Oct 24) re-applies the same check on reschedule/reassignment. Why isn't the procedure enough — what write path bypasses it? Conversely, what breaks if only the trigger existed?

Answer

The booking procedure only protects creating an appointment (INSERT). An appointment's time slot can change after it's booked through a direct UPDATE, because service advisors and managers have UPDATE access on ServiceAppointment, so they can reschedule and reassign appointments (FR-14). Before Oct. 24, an authorized user could reschedule an appointment, extend its duration, or reassign its technician with an UPDATE and create an overlap that the booking procedure would have rejected. The trigger closes that path: whenever an active appointment's time, duration, or technician changes, it re-runs the checks (valid duration, an active technician, and no overlap for the vehicle or technician, excluding the appointment itself).

Since there is no BEFORE INSERT trigger on ServiceAppointment, one write path is still unguarded: a direct INSERT. Regular users cannot use it because no role has INSERT access on the table, so every user booking must go through the procedure. However, root can insert appointments directly and skip both checks.

If only the trigger existed, new bookings would not be checked at all, because it runs on UPDATE, not INSERT. Even as a BEFORE INSERT trigger, it could not fully replace the procedure. A trigger cannot run SELECT … FOR UPDATE on the table that fired it, so my trigger's overlap check reads the existing appointments without locking them and can see an older snapshot that misses a booking another session has just committed. The procedure locks the technician and vehicle rows, so conflicting bookings take turns, and then reads the existing appointments with FOR UPDATE. That read always sees the latest committed data, so two

simultaneous bookings for the same slot cannot both succeed. For the final version, I would route appointment creation, rescheduling and technician reassignment through procedures using the same locking strategy, add FOR SHARE to any overlap reads that remain in triggers, and keep triggers for additional validation.

2. **Failure audit after rollback:** a rejected procedure call still leaves an audit row even though the transaction rolled back. How is that row written so it survives the rollback? Why does the distinction between success-audit and failure-evidence matter?

Answer

When a procedure call fails, its EXIT HANDLER captures the error with GET DIAGNOSTICS and then issues a ROLLBACK, which undoes every change the call made. That includes any success-audit rows, because those are written by AFTER triggers during each statement, inside the same transaction. Only after the rollback does the handler work out the acting user (the logged-in user if their login maps to a UserAccount, otherwise the user ID passed into the procedure) and write a Rejected row through sp_WriteAuditLog. That row stores the procedure name, the error message and the inputs in NewValue, leaves PreviousValue NULL, and carries the call's ContextID. Finally, the handler uses RESIGNAL to re-raise the original error, so the caller still sees the call fail. The rejected row survives because it is written after the rollback, outside the procedure's transaction.

The distinction between success-audit and failure-evidence matters because they record different things. A success-audit row records a change, so it must exist if and only if that change committed. This is guaranteed by writing it inside the same transaction: if a procedure fails after making some changes, the rollback removes those changes and audit rows together so that the audit log never shows a change that did not happen. A rejected row records an attempt that changed nothing, so it has to be written outside of the transaction, otherwise the rollback would erase the evidence that the attempt happened. If either kind were written the other way, the log could not be trusted: it would either lose failed attempts or show changes that never happened. Rejected rows are marked as (Action = 'Rejected') so that anyone rebuilding a record's history can skip them, while they still show who tried to do what, with which inputs, and why it was refused. This is useful for security, debugging and accountability.

This approach has limitations. Only the procedures write Rejected rows: when a trigger rejects a direct INSERT or UPDATE, the whole statement is rolled back and nothing is recorded. If a caller runs a procedure inside its own open transaction, the Rejected row becomes part of that transaction and the caller's rollback would remove it. If the call is refused before it starts, for example because the user lacks EXECUTE permission, the handler never runs, so no Rejected row is written. In the final version, I plan to route all writes through procedures, so that every rejected change leaves evidence.

3. **Full-row JSON audit vs. column-level:** you store the entire row as JSON in the audit log. What query does that make easy, and what query does it make painful (e.g., "every change to appointment status last month")? How do you index into it?

Answer

Storing the full row as JSON makes record history easy. To see everything that happened to one appointment, I filter on AffectedEntity and AffectedRecordID and sort by Timestamp, which matches my index idx_AuditLog_Entity_Record (AffectedEntity, AffectedRecordID, Timestamp). Because every Update row stores every non-key column before and after, each row is a complete snapshot. I can see what the record looked like at any point, even after it was deleted.

The painful queries ask about one column across many records over time. My index is sorted by table, then by record ID, then by time. A query such as "every change to appointment status last month" filters on AffectedEntity and Timestamp but not AffectedRecordID, so MySQL can use AffectedEntity to jump to the ServiceAppointment rows but can only use Timestamp to filter them: that month's rows are spread across every appointment, because Timestamp is sorted only within each record, so MySQL has to go through every ServiceAppointment audit entry and check each timestamp. Status changes are partly handled because my audit trigger labels them in the Action column as Status Change, Cancel, or Complete, so that query only needs to filter on Action and Timestamp, without opening the JSON. Other columns are harder. Action holds only one label per update, and Status takes priority, so an appointment that is rescheduled and changes status in the same update is labeled Status Change. Therefore, finding every reschedule means opening the JSON in every row and

comparing the old and new AppointmentDateTime, which no index can help with because the values are inside the JSON.

To index into it, I would add an index on (AffectedEntity, Action, Timestamp) so that a time range for a given action, such as status changes last month, is one block MySQL can jump to instead of scanning. MySQL cannot index a JSON column directly, so for fields that are queried often, such as the appointment time for finding reschedules, I would extract them into generated columns and index those. Each one covers only one field, so I would add them only for questions that are actually asked. I chose full row JSON because record history was my main audit requirement.

4. **Trigger-heavy design:** 1,173 lines of triggers vs. two procedures. Defend the split — which rules belong in triggers, which in procedures, and what is the debugging cost when a trigger chain misfires mid-transaction?

Answer

About half of the trigger file is audit, not business rules: 22 AFTER triggers that each serialize one table's columns to JSON, which is repetitive by nature. The other half is validation. I split the rules by one principle. Triggers enforce invariants, facts that must hold no matter who writes the data or how: allowed status transitions, IDs and creating users that never change, finalized estimates and invoices that cannot be edited, totals that must equal the sum of their line items, approval fields that only the approval itself can set, and the audit trail. The procedures handle workflows: user actions that need inputs the row does not contain, coordinate changes across several tables as one unit, need to control the full locking order, or must record a Rejected row on failure. This is why booking an appointment and completing a repair are procedures.

The design leans on triggers because my roles still have some direct table privileges (service advisors and managers can update ServiceAppointment, managers can insert and update Part), and an administrator can write to any table. With those paths open, triggers were the only layer guaranteed to see every write.

The debugging cost is real. A single UPDATE can fire a BEFORE trigger, an AFTER trigger and sp_WriteAuditLog, and the caller only sees the final error message with SQLSTATE 45000. MySQL does not report which trigger raised it or show the chain, and the whole statement rolls back. A trigger-raised error also leaves no Rejected row, and trigger order has to

be managed explicitly with FOLLOWS. I contain this by giving every rule a distinct error message, so the message identifies the rule, by having procedures capture the error with GET DIAGNOSTICS and record it with the inputs in a Rejected row, and by testing each rule individually in 07_tests.sql.

For the final version, I plan to make every write go through procedures, with roles granted EXECUTE only and no direct table privileges. Status changes will go through one procedure driven by a transition table, and triggers will be reduced to the audit and a few last-line invariants, such as locking finalized records. That gives every rejected action a Rejected row and makes procedures the place where rules are read and debugged.

5. **Status completion rule:** an appointment must be "In Progress" to complete a repair, and cannot close with open repairs (both Oct 24 additions). Show the state machine — and one transition your design *rejects* that a real shop might legitimately need.

Answer

State Machine

Appointment: Scheduled → Checked In → In Progress → Completed

Scheduled → Cancelled | No Show

Repair: Open → In Progress → Completed

Open | In Progress → Cancelled

The appointment state machine is enforced by trg_ServiceAppointment_Status_Validate. An appointment booked through the stored procedure starts as Scheduled and can move to Checked In, Cancelled, or No Show; Checked In moves only to In Progress, and In Progress moves only to Completed. Completed, Cancelled, and No Show are final states that cannot be modified. Repairs have their own state machine in trg_Repair_Status_Validate: a repair starts Open (and cannot be added to a closed appointment), Open moves to In Progress or Cancelled, In Progress moves to Completed or Cancelled, and Completed and Cancelled are final. The two rules added on Oct. 24 connect the two machines. The appointment trigger rejects closing an appointment (Completed, Cancelled, or No Show) while any of its repairs is still Open or In Progress. sp_CompleteRepair rejects completing a repair unless its appointment is In Progress.

That second rule lives in the procedure so it does not apply to the documented manager override that completes a repair with a direct update.

One legitimate transition my design rejects is Checked In → Cancelled. A customer drops off the car, the technician inspects it, and the customer declines the estimate and takes the car home. From Checked In, the only allowed move is In Progress, so to close the appointment, staff would have to move it to In Progress, cancel its repairs, and mark it Completed, which records work that never happened, even though the estimate itself can be marked Rejected. The fix is to allow Checked In → Cancelled and In Progress → Cancelled once every repair is completed or cancelled, which the close rule already enforces. In the final version, I plan to move these transitions into a table checked by a single procedure, so adding a transition like this one becomes a new row rather than a trigger change.

6. **Performance work:** OPT-01's covering index — show the before/after EXPLAIN and timings, and the data volume behind them. At what row count does the un-indexed version actually fall over?

Answer

OPT-01 was measured on 20,008 Repair rows (10,004 Completed) generated by perf_data.sql. Before the index, EXPLAIN ANALYZE showed a full table scan of all 20,008 rows, a filter on Status, and an aggregate using a temporary table, for a total of 7.84 ms. After adding the index on (Status, RepairType), the plan became a covering index lookup that reads only the 10,004 Completed index entries and groups them with no temporary table for a total of 2.21 ms, about 3.5 times faster. The full output is in docs/performance/OPT-01_output.txt.

At 20,000 rows, the un-indexed version does not fall over. To find where it does, I ran the same query on a standalone copy of the Repair table with the same data distribution at increasing sizes (docs/performance/OPT-01_scale_test.sql, with output in OPT-01_scale_output.txt). Without the index, it took 20.4 ms at 100,000 rows, 121 ms at 500,000, 227 ms at 1 million, and 1.14 s at 5 million; with the index, 9.4 ms, 49 ms, 98 ms, and 488 ms. Both grow linearly. The index gives an improvement of about 2.3 times but not a different growth rate because the query must count every Completed repair, which is half the table; the index only lets MySQL skip the other half and avoid the temporary table. For a 100 ms target, the un-indexed version crosses it

between 100,000 and 500,000 rows, and the indexed version at about 1 million. Neither fails outright; both get slower as the data grows.

7. **Solo scope:** you built this alone. What did you deliberately cut, and if you had a second member, what would you add first?

Answer

Working alone, I focused on making the core workflows correct under concurrency, fully audited and tested, with 66 automated checks in 07_tests.sql. I left out the user interface since the next phase of the course adds one, along with items listed as out of scope in the FRD: real payment processing, integrations with external scheduling systems, supplier and vehicle-history systems, a customer portal and AI diagnostics. Other cuts are documented as known limitations in the TDD. Finalized invoices cannot be voided or corrected. Master-data changes to Customer, Vehicle, Technician and Part details are not audited. Approved estimates can still be edited until they are finalized. Row-level filtering for technicians covers appointments and repairs but not vehicles or parts. Only the two most complex workflows, booking an appointment and completing a repair, are stored procedures. Other writes rely on direct table privileges protected by triggers.

With a second member, I would first complete the procedure layer. Every write would go through a stored procedure, the UI's account would be granted EXECUTE only, and status changes would go through one procedure driven by a transition table. This matters because the UI we build next will connect through one service account on behalf of many users, so it should only be able to do what the procedures allow rather than writing to tables directly. The procedures would record both the service account and the real user in the audit log and every rejected action would leave a Rejected row. It would also put rules like the overlap check in one place instead of two.

8. **Line-total invariant:** "line totals must equal quantity × unit price" (Oct 25). Where is it enforced, and what happens on a price change after lines exist?

Answer

The invariant is enforced by CHECK constraints in 01_schema.sql. chk_EstimateLineItem_Total and chk_InvoiceLineItem_Total both require LineTotal = Quantity

* UnitPrice. MySQL checks them on every insert and update of a line item, from any write path, so a line with a wrong total cannot be saved. Because a CHECK can only see one row, triggers handle the rules that involve other rows: the estimate or invoice total must equal the sum of its line items when an estimate or invoice is finalized, and once a document is finalized, its line items cannot be added, changed, or deleted.

A change to a part's price does not affect existing lines. Each line item stores its own UnitPrice, set when the line was written, with no link to the Part table, and PartsUsage stores UnitSellingPriceAtTimeOfUse. Estimates and invoices therefore keep the price the customer was given. Editing a line's price before the document is finalized is rejected unless LineTotal changes in the same statement, and the document total must be corrected before finalization.

The main weakness is that LineTotal is stored rather than computed. Making it a generated column would make a mismatch impossible, and I plan to do that for the final. The document total is also checked only at finalization, so a draft can be inconsistent in between.